

```
"""
```

```
technical.py
```

```
-----
```

```
Computes technical trend/momentum/volatility indicators and detects  
market structure (higher-highs/higher-lows vs lower-highs/lower-lows,  
or range-bound) from OHLC data.
```

```
Everything reduces to a single 'TechnicalScore' in roughly [-1, +1]:
```

```
+1 = strong, clean bullish structure for the base currency
```

```
0 = no clear trend / mixed signals
```

```
-1 = strong, clean bearish structure for the base currency
```

```
"""
```

```
import pandas as pd
```

```
import numpy as np
```

```
def ema(series: pd.Series, span: int) -> pd.Series:  
    return series.ewm(span=span, adjust=False).mean()
```

```
def rsi(series: pd.Series, period: int = 14) -> pd.Series:  
    delta = series.diff()  
    gain = delta.clip(lower=0)  
    loss = -delta.clip(upper=0)  
    avg_gain = gain.ewm(alpha=1 / period, adjust=False).mean()  
    avg_loss = loss.ewm(alpha=1 / period, adjust=False).mean()  
    rs = avg_gain / avg_loss.replace(0, np.nan)  
    return (100 - 100 / (1 + rs)).fillna(50)
```

```
def macd(series: pd.Series, fast: int = 12, slow: int = 26, signal: int = 9):  
    macd_line = ema(series, fast) - ema(series, slow)  
    signal_line = ema(macd_line, signal)  
    hist = macd_line - signal_line  
    return macd_line, signal_line, hist
```

```
def atr(df: pd.DataFrame, period: int = 14) -> pd.Series:  
    high, low, close = df["High"], df["Low"], df["Close"]  
    prev_close = close.shift(1)  
    tr = pd.concat([  
        high - low,  
        (high - prev_close).abs(),
```

```

        (low - prev_close).abs(),
    ], axis=1).max(axis=1)
    return tr.ewm(alpha=1 / period, adjust=False).mean()

def find_swings(series: pd.Series, window: int = 5) -> pd.DataFrame:
    """
    Identifies swing highs/lows using a rolling-window local extremum
    check: a point is a swing high if it's the max within +/- window bars,
    a swing low if it's the min within +/- window bars.
    """
    highs, lows = [], []
    vals = series.values
    for i in range(window, len(vals) - window):
        seg = vals[i - window:i + window + 1]
        if vals[i] == seg.max():
            highs.append((series.index[i], vals[i]))
        if vals[i] == seg.min():
            lows.append((series.index[i], vals[i]))
    return highs, lows

def classify_structure(df: pd.DataFrame, window: int = 5) -> dict:
    """
    Looks at the sequence of recent swing highs and swing lows to classify
    market structure as 'uptrend' (HH+HL), 'downtrend' (LH+LL), or 'range'.
    """
    highs, lows = find_swings(df["Close"], window=window)

    structure = "range"
    hh = hl = lh = ll = False

    if len(highs) >= 2:
        hh = highs[-1][1] > highs[-2][1]
        lh = highs[-1][1] < highs[-2][1]
    if len(lows) >= 2:
        hl = lows[-1][1] > lows[-2][1]
        ll = lows[-1][1] < lows[-2][1]

    if hh and hl:
        structure = "uptrend"
    elif lh and ll:
        structure = "downtrend"

    return {
        "structure": structure,
        "n_swing_highs": len(highs),
    }

```

```

    "n_swing_lows": len(lows),
    "higher_highs": hh,
    "higher_lows": hl,
    "lower_highs": lh,
    "lower_lows": ll,
}

```

```

def compute_technical_signals(df: pd.DataFrame) -> dict:
    close = df["Close"]

    ema20, ema50, ema200 = ema(close, 20), ema(close, 50), ema(close, 200)
    rsi14 = rsi(close, 14)
    _, _, macd_hist = macd(close)
    atr14 = atr(df, 14)
    struct = classify_structure(df)

    last_close = close.iloc[-1]
    last_ema20, last_ema50 = ema20.iloc[-1], ema50.iloc[-1]
    last_ema200 = ema200.iloc[-1] if len(df) >= 200 else np.nan
    last_rsi = rsi14.iloc[-1]
    last_macd_hist = macd_hist.iloc[-1]
    last_atr = atr14.iloc[-1]

    # --- Trend score from EMA stack ---
    trend_score = 0.0
    if last_close > last_ema20 > last_ema50:
        trend_score += 0.5
    elif last_close < last_ema20 < last_ema50:
        trend_score -= 0.5
    if not np.isnan(last_ema200):
        trend_score += 0.5 if last_close > last_ema200 else -0.5

    # --- Momentum score from RSI + MACD histogram ---
    momentum_score = 0.0
    momentum_score += np.clip((last_rsi - 50) / 25, -1, 1) * 0.5
    momentum_score += np.clip(last_macd_hist / (last_atr + 1e-9), -1, 1) * 0.5

    # --- Structure score from swing pattern ---
    structure_score = {"uptrend": 1.0, "downtrend": -1.0, "range": 0.0}[struct["s

technical_score = np.clip(
    0.4 * trend_score + 0.3 * momentum_score + 0.3 * structure_score, -1, 1
)

return {
    "LastClose": round(float(last_close), 5),

```

```

    "EMA20": round(float(last_ema20), 5),
    "EMA50": round(float(last_ema50), 5),
    "EMA200": None if np.isnan(last_ema200) else round(float(last_ema200), 5)
    "RSI14": round(float(last_rsi), 1),
    "MACD_Hist": round(float(last_macd_hist), 6),
    "ATR14": round(float(last_atr), 5),
    "TrendScore": round(float(trend_score), 2),
    "MomentumScore": round(float(momentum_score), 2),
    "StructureScore": round(float(structure_score), 2),
    "Structure": struct["structure"],
    "TechnicalScore": round(float(technical_score), 3),
}

```

```

if __name__ == "__main__":
    from data_feed import generate_synthetic_ohlc
    df = generate_synthetic_ohlc(n=220, seed=42, drift=0.0006)
    signals = compute_technical_signals(df)
    print("=== Technical Signals (synthetic test data) ===")
    for k, v in signals.items():
        print(f"{k:15s}: {v}")

```